Milestone-3 Design Doc

Demo Setup + Execution

To replicate demo results:

Requirements

- Docker Desktop (Mac/Windows) or Docker Engine (Linux)
- At 10 GB disk space recommended
- Provided Docker image

Quick start

```
# Step 1: build the image (may take 30+ min)
docker compose build
# Step 2: run the benchmark
docker compose run comet-demo bash /opt/scripts/run
```

What the build does

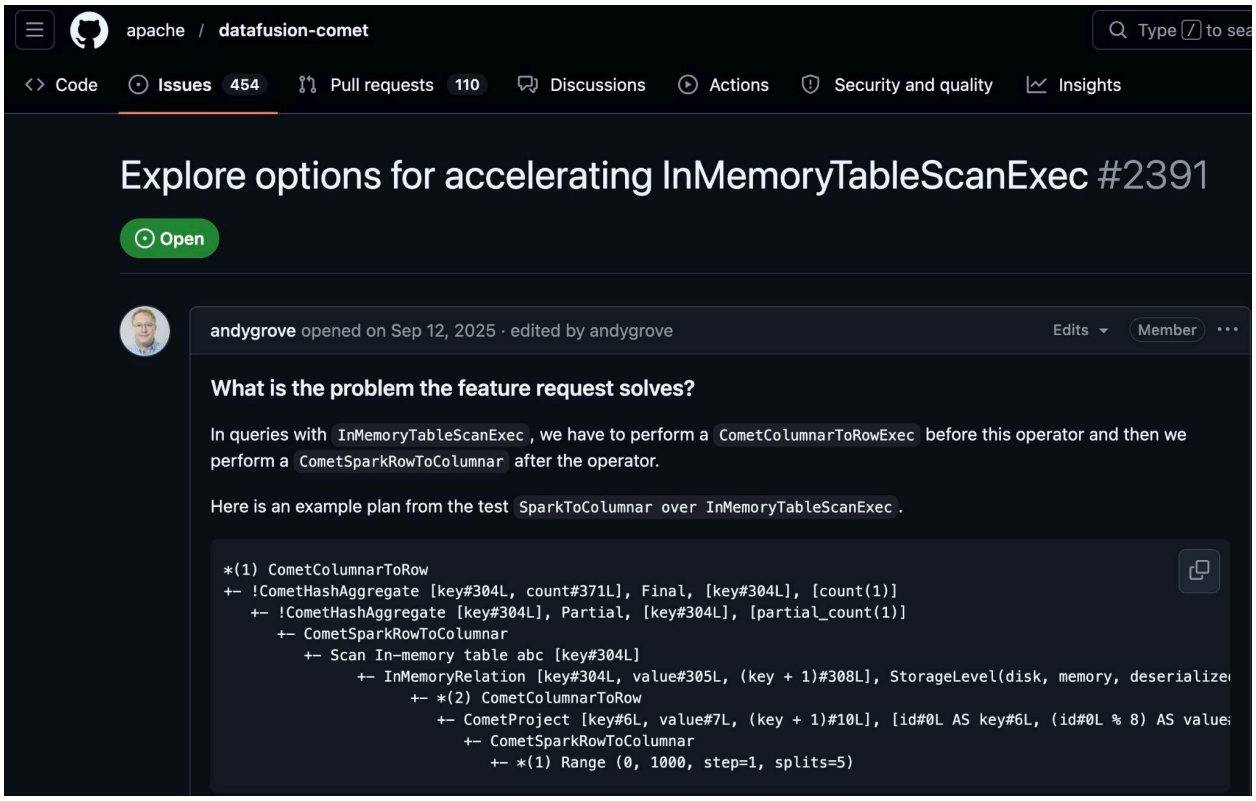
1. Installs Java 17, Rust, Maven
2. Downloads Spark 3.5.8
3. Builds Comet from source

For the Milestone-3, we focused on non-benchmark related operator and sql function optimization by providing native comet support to certain operations that were previously unsupported and used to fall back to spark's JVM. We chose non-benchmark related issues for milestone-3 primarily due to:

1) There are several senior Apache Datafusion engineers already working exactly on what we planned to work on w.r.t the benchmarks(TPC-H, TPC-DS) based operations and their commits are very rapid and directly solved several of the problems we wanted to solve ourselves very recently.
2) Legacy benchmarks such as TPC-H & TPC-DS are at least 15 years and so the queries provided in the benchmark are not always the most optimal way to get results especially with the recent advancements in new aggregate functions w.r.t sophistication & efficiency.

So, we focused on Github issues that were not directly related to the benchmark query optimization for this milestone. Below is the first one:

Milestone-3 Design Doc



Official GitHub issue link: [Explore options for accelerating InMemoryTableScanExec #2391]

Our Solution

Eliminating Spark↔Comet Conversion Overhead

The core mechanism is to **use a Comet-controlled, Arrow-based columnar cache representation directly during reads**, instead of routing cached data through Spark's default row-oriented in-memory scan path.

This removes the need for repeated row↔columnar conversions between Spark and Comet.

Milestone-3 Design Doc

Approach

Our solution consists of two coordinated components:

1. Cache Write Path

Cached data is stored in a **Comet-specific format backed by serialized Arrow batches**, rather than Spark's default cached-batch representation.

- Input data (row or columnar) is converted into **Arrow columnar batches**
- These batches are serialized and stored as **CometCachedBatch**
- This ensures cached data is already in a **columnar, Comet-compatible form**

---

2. Cache Read Path

Cached data is read through a **Comet-specific scan operator**, bypassing Spark's default `InMemoryTableScanExec`.

- Cached payload is **decoded directly into columnar batches**
- Output is immediately usable by Comet operators
- No intermediate row-based processing occurs

During read, only the **columns required by the query** are selectively materialized from the cache data to reduce unnecessary data movement

---

Next, for SQL aggregation functions **`count_if` and `mode`**, we implemented them as native aggregates in Comet so they execute without falling back to Spark as they're currently unsupported by Comet. `count_if` maintains a **single running counter** that increments when the condition is true, making it simple and constant-memory. In contrast, `mode` tracks a **frequency table (value → count)** to determine the most frequent value, which requires maintaining structured state. To support this within Comet's execution model, the frequency table is stored as a **single compact state representation**, allowing it to be passed and combined correctly across execution stages. This ensures both functions run fully in Comet while preserving correctness and efficiency.